Data Structures & Systems Design

AI / Multi-Agent Systems Intern Take-Home

Candidate: Akshay Mehta
Company: Avaloka (Guruvai Sciences)
Date: October 2026

Problem 1: LRU Cache Implementation

Logic & Approach Explanation

To fulfill the requirement of **O(1) time complexity** for both `get` and `put` operations, we pair a **HashMap** with a custom **Doubly Linked List (DLL)**:

- **HashMap<Integer, Node>:** Stores key-to-node references, giving constant time O(1) lookup.
- **Doubly Linked List:** Maintains recency order. Accessed or added nodes are placed at the front (after dummy `head`), while the least recently used item sits before dummy `tail`.
- **Why Doubly Linked?** In a singly linked list, removing a node requires an O(N) traversal to locate its predecessor. Direct `prev` and `next` pointers allow unlinking nodes in O(1) time.
- **Sentinel Nodes:** Dummy `head` and `tail` eliminate null boundary checks during insertions and deletions.

```
    +-----------------------------------------------------------+ | HashMap | | Key "A" --> Node A | | Key "
```

Java Source Code

```
      package com.avaloka.assignment; import java.util.HashMap; import java.util.Map; public class L
```

Problem 2: Event Scheduler

Algorithmic Logic Explanation

The event scheduling problem requires evaluating interval overlaps and dynamic resource consumption:

- **canAttendAll:** Sort events chronologically by start time. An overlap occurs if any event starts *strictly before* the preceding event concludes (`currentStart < previousEnd`). Strict inequality allows adjacent events like `(9, 10)` and `(10, 11)` without overlap.
- **minRoomsRequired:** Sort events by start time and process them through a **Min-Heap (PriorityQueue)** storing active meeting end times. For each event `(start, end)`, if `heap.peek() <= start`, the room is freed and popped from the heap. The current meeting's end time is then pushed. The maximum size of the heap gives the minimum rooms required.

```
Input Events: [(9, 10), (9, 12), (10, 11), (11, 12)] | v [Sort by Start Time] --> [(9, 10), (9, 12), (
```

Java Source Code

```
package com.avaloka.assignment; import java.util.Arrays; import java.util.Comparator; import j
```

Final Discussion & Analysis

1. Time & Space Complexity Analysis

Function	Time Complexity	Space Complexity	Complexity Rationale
LRUCache.get	O(1)	O(1) auxiliary	Hash map lookup and DLL pointer re-linking operate in constant time.
LRUCache.put	O(1)	O(1) auxiliary	Node eviction, insertion, and map updates run in constant time. Bounded by O(C) capacity space.
canAttendAll	O(N log N)	O(1) / O(N)	Sorting N events takes O(N log N). Single linear scan takes O(N) time.
minRoomsRequired	O(N log N)	O(N) auxiliary	Sorting takes O(N log N). PriorityQueue heap operations cost O(log N) per event across N events.

2. Trade-off Analysis: Hash Map + Doubly Linked List

Neither an array, hash map, nor linked list individually fulfills the O(1) performance requirement:

- **Array alone:** Direct indexing is O(1), but searching keys is O(N) and element shifts cost O(N).
- **HashMap alone:** Key lookups are O(1), but it lacks access ordering. Finding the LRU key requires scanning all entries in O(N) time.
- **Doubly Linked List alone:** Maintains recency order and enables O(1) unlinking, but search costs O(N).
- **Trade-off Choice:** Combining both trades extra memory (storing explicit `prev` and `next` node pointers) to achieve optimal **O(1) runtime execution** for both operations.

3. Future Proofing: Specific Room Assignment ("Room A", "Room B")

To assign human-readable room identifiers (e.g., "Room A", "Room B"), maintain an active allocation heap alongside a pool of recycled room names:

- **Name Recycling Pool:** Store freed room names in a `PriorityQueue<String>` pool.
- **Active Allocation Heap:** Store tuples of `(end_time, assigned_room)` sorted by end time.
- **Logic:** When a new event starts, pop all expired meetings from the heap and add their room names to the pool. Assign a room from the pool if available; otherwise, instantiate a new room name ("Room " + ++counter).

4. Concurrency & Thread Safety

In multi-threaded environments, concurrent calls to `get` and `put` introduce race conditions on linked list pointers and HashMap entries.

- **ReentrantLock:** Wrap `get` and `put` with an explicit lock to ensure atomic operations.
- **Read/Write Lock Constraint:** Standard `ReentrantReadWriteLock` cannot treat `get()` as a read operation because `get()` performs structural write mutations (moving nodes to MRU position).
- **Cache Sharding:** To reduce lock contention under high concurrency, partition keys across N independent shards based on `Math.abs(key.hashCode()) % N`.